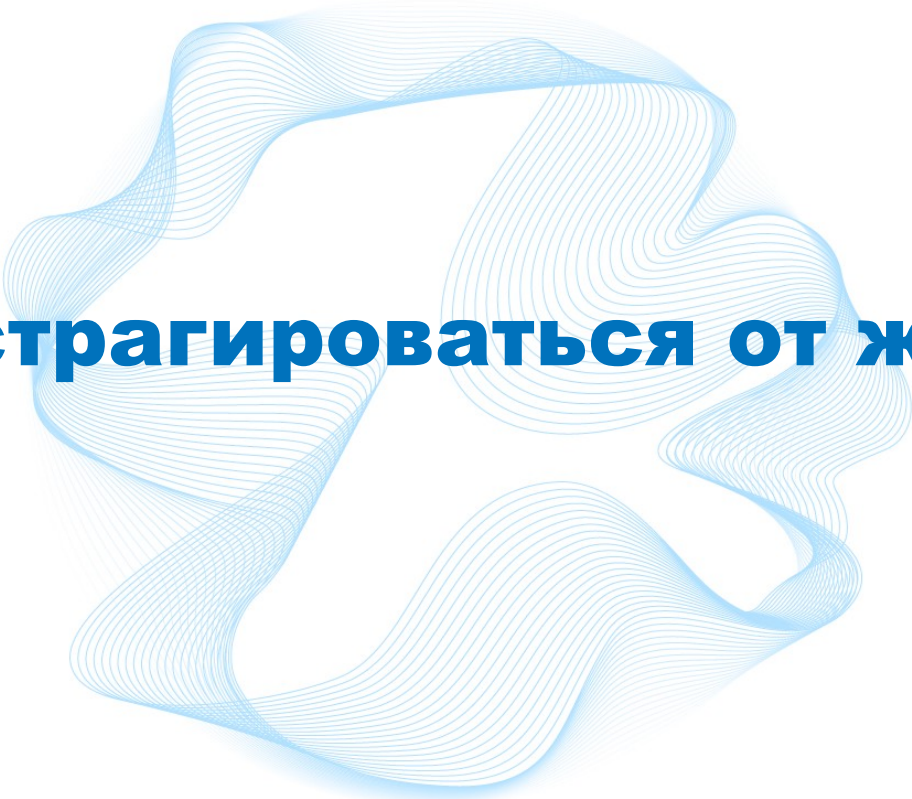




Как абстрагироваться от железа?

Цель занятия

- Овладеть навыком абстракции при разработке встраиваемых систем

Внешняя периферия микроконтроллера

Абстракция

Абстрагирование — операция мышления, состоящая в отвлечении от несущественных сторон, свойств, связей объекта (предмета или явления) с целью выделения их существенных, закономерных признаков.

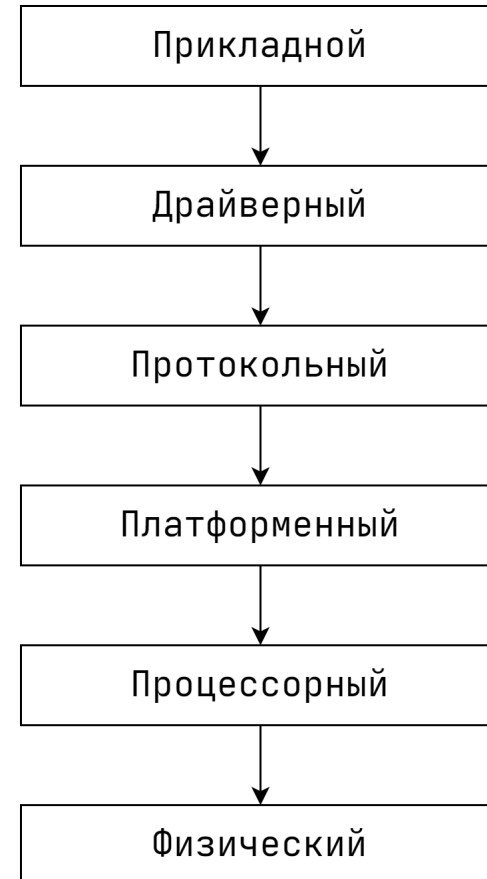


Абстракция

Абстракция во встроенных системах — принцип, согласно которому некоторая часть системы берёт на себя ответственность по реализации определённого функционала и скрывает детали его реализации, предоставляя пользователю интерфейс для использования этого функционала

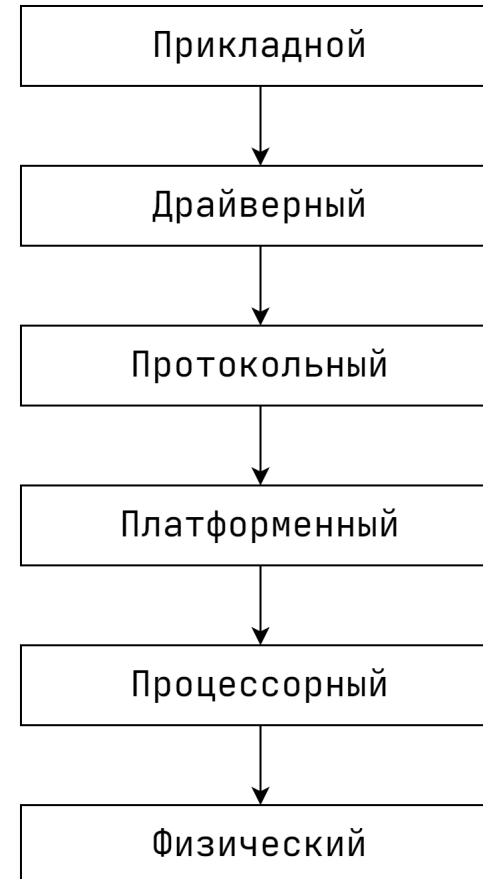
Уровни абстракции

Уровни абстракции во встроенных системах — представление системы как иерархически связанных абстрактных частей, каждая из которых использует функционал частей нижнего уровня и не имеет доступа к их деталям реализации.



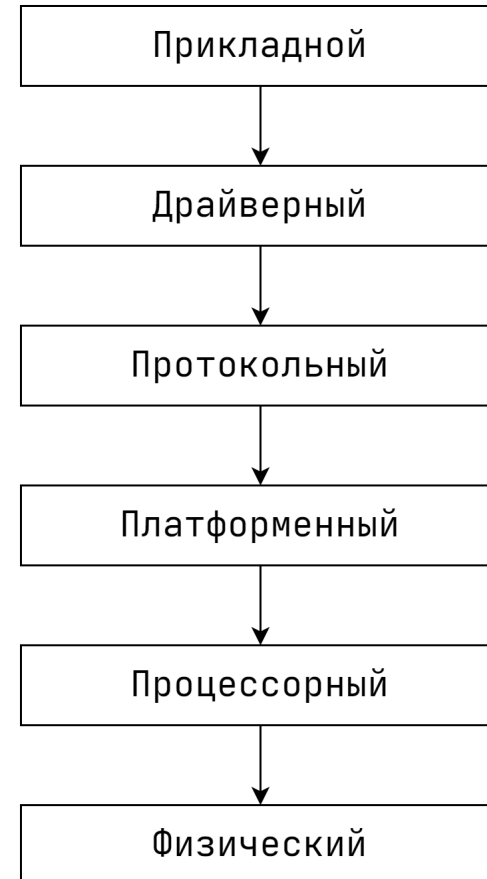
Физический

Ответственность: корректность подключений контактов, передачи сигналов, согласование напряжений (3,3 В), ёмкостей шины, подтяжек. К этому уровню относятся и механические параметры: влагозащищённость корпуса, диапазон температур, виброустойчивость.



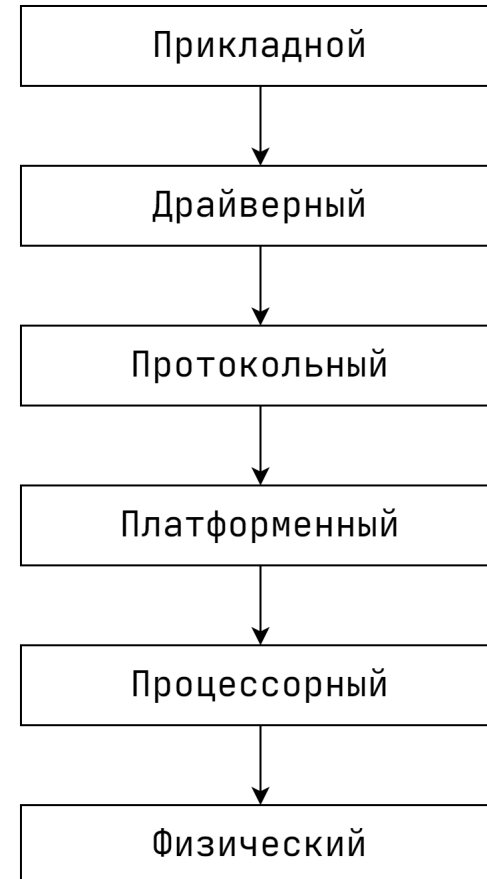
Процессорный

Ответственность: корректное исполнение программы на конкретном процессорном ядре. Мало кто задумывается при программировании на ПК или Arduino, какой процессор выполняет инструкции, какая у него разрядность, сколько регистров и какой размер кэша — все эти детали скрыты.



Платформенный

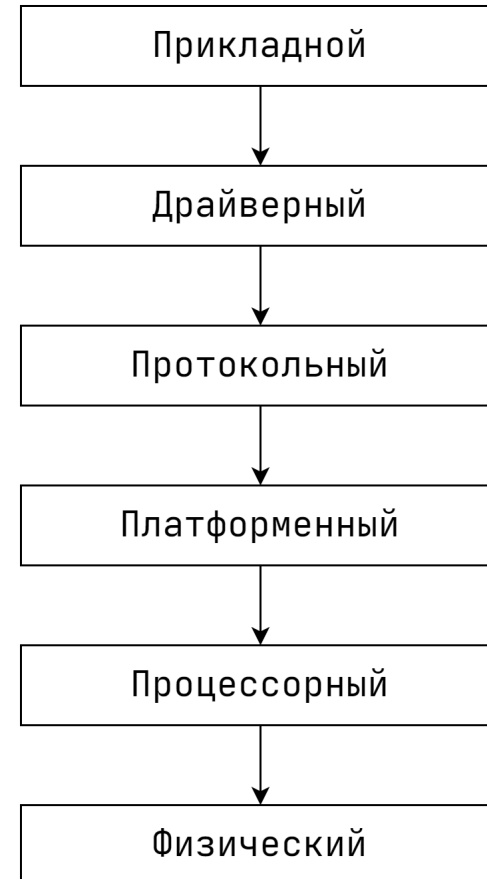
Ответственность: корректное использование периферии микроконтроллера — SPI, I2C, UART, таймеры, GPIO. Когда нужен SPI, вы не хотите знать, в какие регистры писать и какие биты выставлять. Вы хотите выполнить транзакцию и передать буфер байтов. Максимум, что вы указываете — идентификатор периферии (`i2c0`, `spi0`) и параметры (адрес устройства, частота).



Протокольный

Ответственность: интерпретация бит данных, как более абстрактных транзакций данных: команды, запись регистров, чтение регистров.

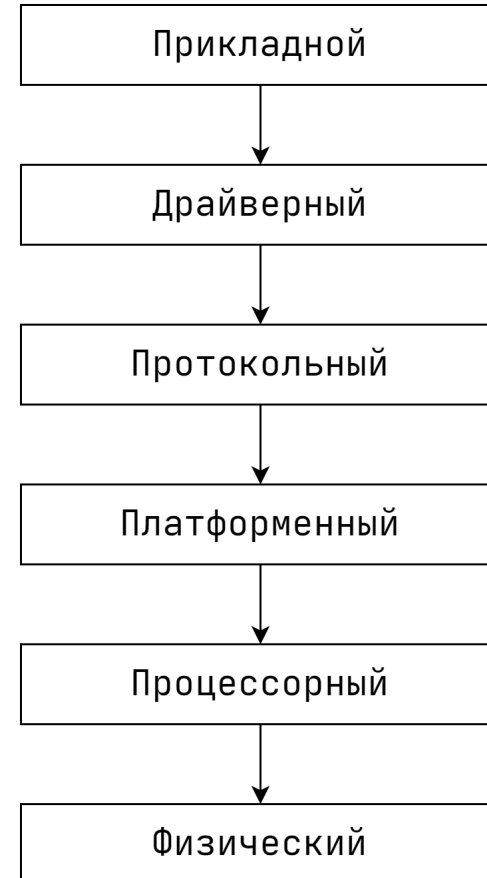
Но он не знает, какие конкретно команды или регистры имеются у устройства.



Драйверный

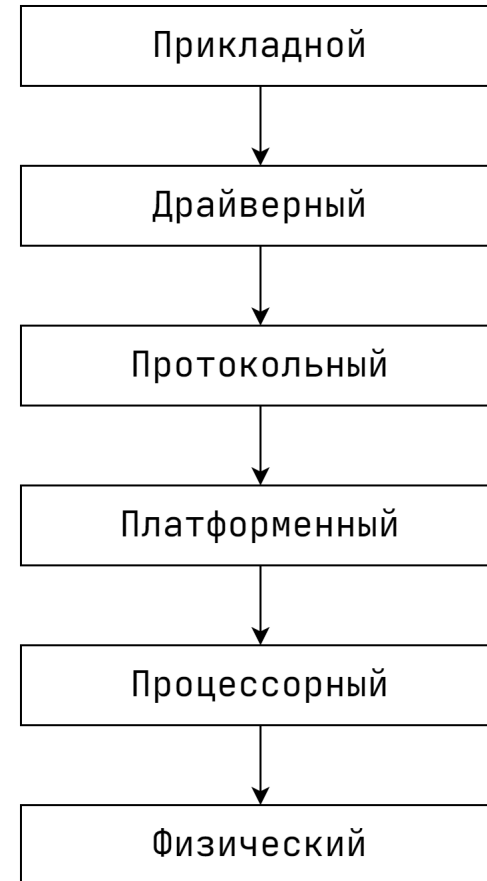
Ответственность: предоставление доступа к функционалу аппаратного устройства при помощи взаимодействия с ним по определенному протоколу.

Этот уровень знает какие регистры и команды есть у устройства.



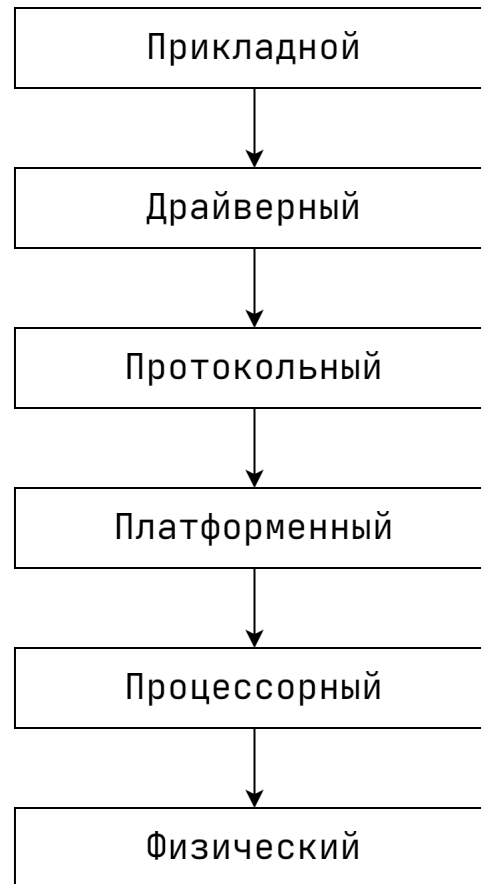
Прикладной

На этом уровне мы окончательно отвлекаемся от аппаратной части. Прикладной код формулирует **что** нужно сделать: «раз в секунду прочитать температуру и отобразить на дисплее», «при нажатии кнопки переключить режим», «отправить данные на ПК по USB». Он вызывает функции драйверов и не обращается к регистрам, SPI или I2C напрямую.



Подходы к работе

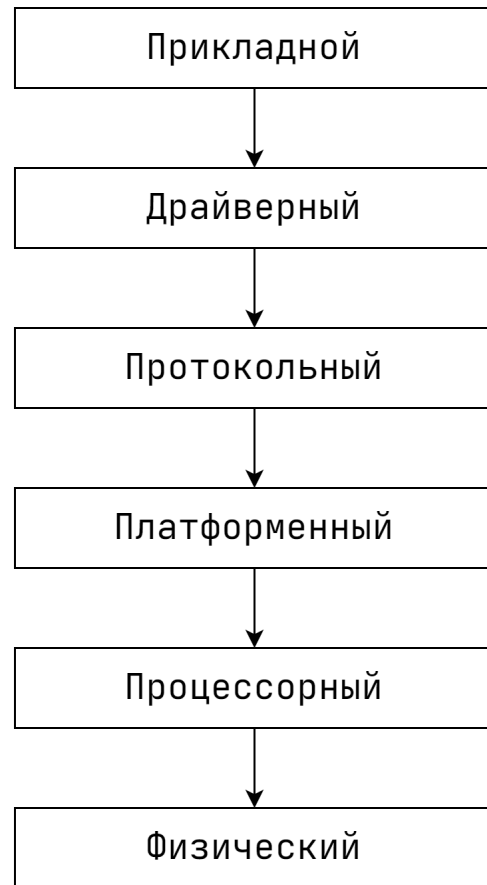
- Поэтапные разработка по уровням
- Работа с интерфейсами
- Распределение ответственности по уровням
- Замена уровней



Сторонние библиотеки

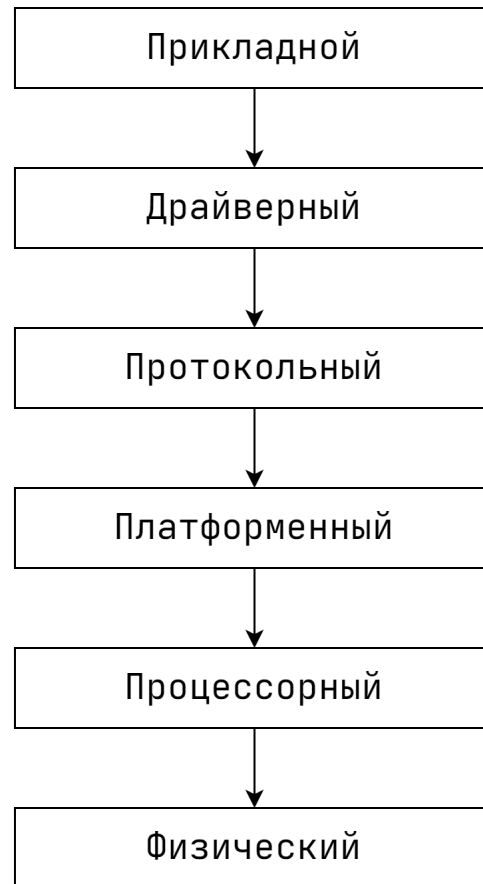
Сторонняя библиотека (*third-party library*) — это программный компонент, который создаётся внешними разработчиками и предоставляет готовые функции на одном или нескольких уровнях абстракции системы.

Контракт библиотеки (library contract) — согласованный набор ожиданий между разработчиком приложения и автором библиотеки: API, документация, ограничения, версия, поведение при ошибках.



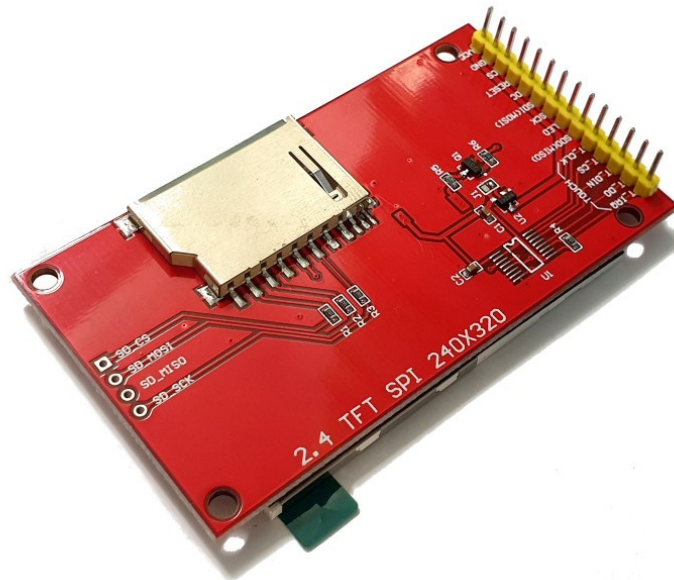
Интеграция сторонней библиотеки

- сформулировать требования
- оцените совместимость
- проверьте контракт
- собрать минимальный пример
- интегрируйте библиотеку



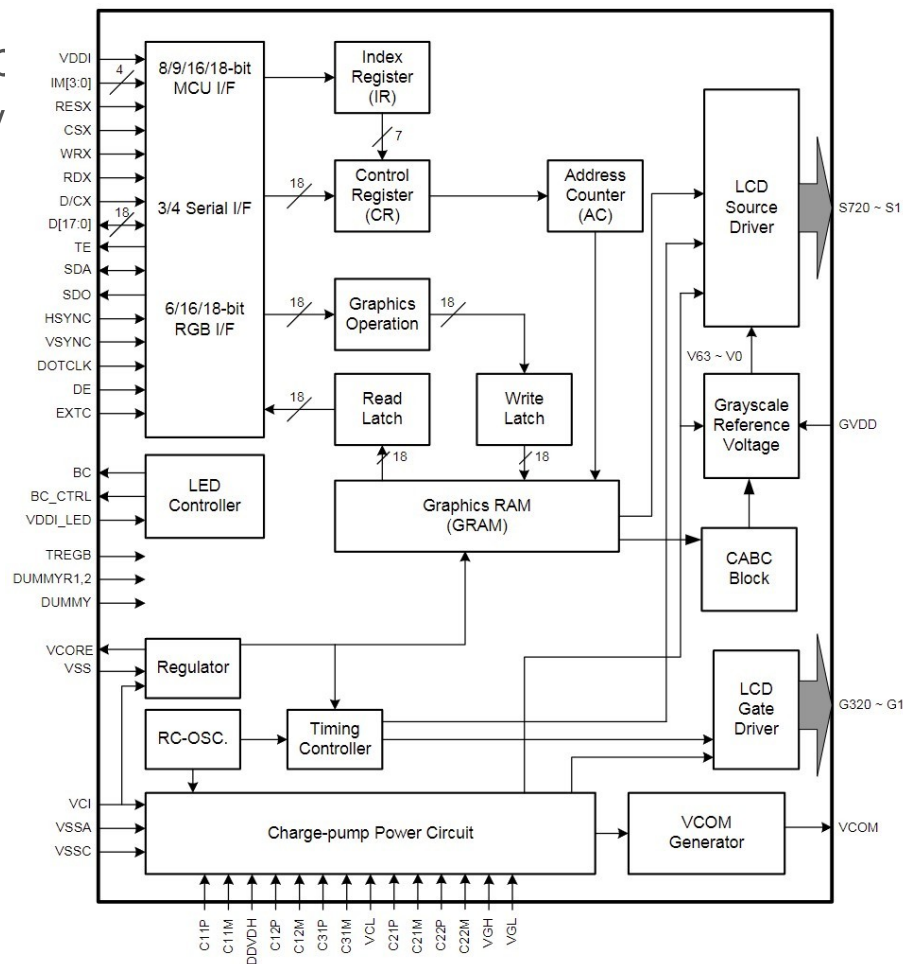
Дисплей

TFT LCD (Thin Film Transistor Liquid-Crystal Display) тонкоплёночный транзисторный жидкокристаллический дисплей 2.4 дюйма 240x320



ILI9341

ILI9341 — однокристальный драйвер TFT LCD производства ILI Technology Corp. Он управляет матрицей пикселей разрешением 240×320 точек и поддерживает до 262 144 цветов (18 бит на пиксель).

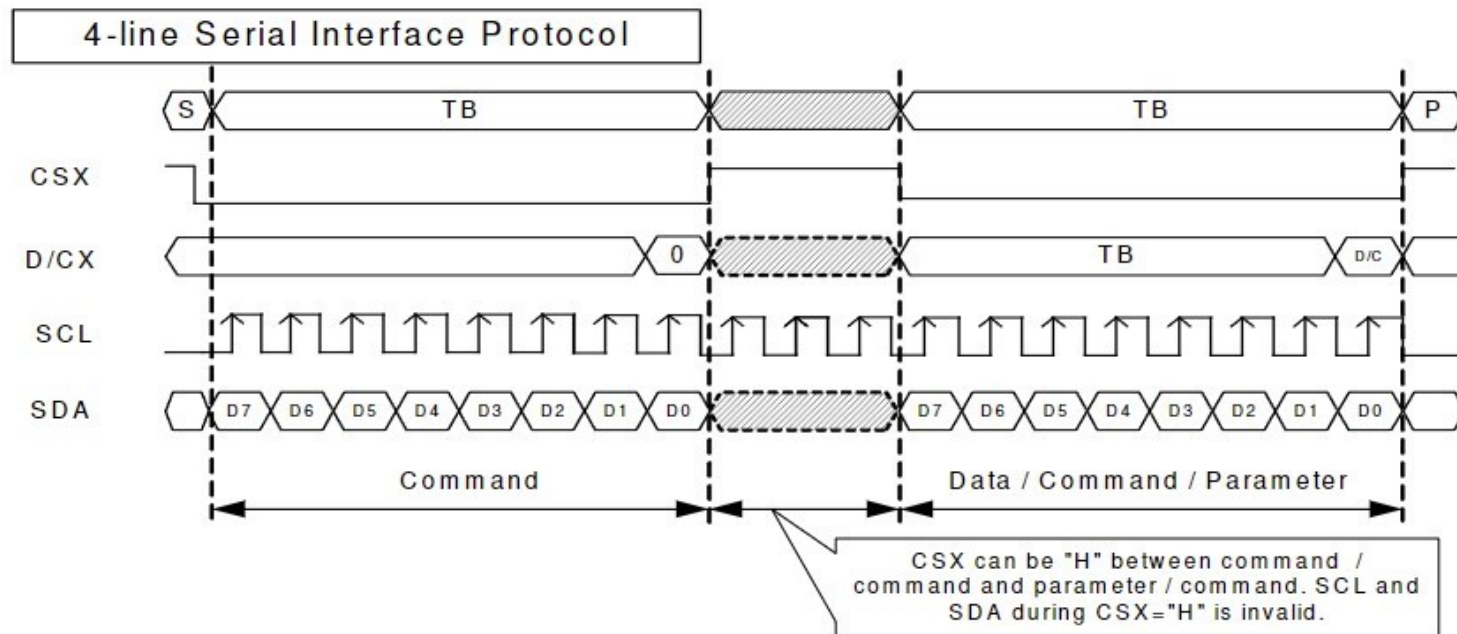


Характеристики ILI9341

- **Разрешение:** 240×320 пикселей (RGB)
- **Глубина цвета:** до 262144 цветов (18 бит, формат RGB 6-6-6) или 65536 цветов (16 бит, формат RGB 5-6-5)
- **Встроенная память (GRAM):** 172800 байт для хранения данных изображения (240×320 точек)
- **Интерфейсы подключения:**
 - последовательный (SPI): 3- или 4-проводной
 - параллельный: 8-, 9-, 16- или 18-битная шина для MCU (например, серии 8080)
 - RGB-интерфейс: 6-, 16- или 18-битный
- **Напряжение питания:** 3,3 В (логика), совместим с 5 В системами при использовании преобразователей уровня
- **Диагональ экрана (в готовых модулях):** обычно 2,4–3,2 дюйма

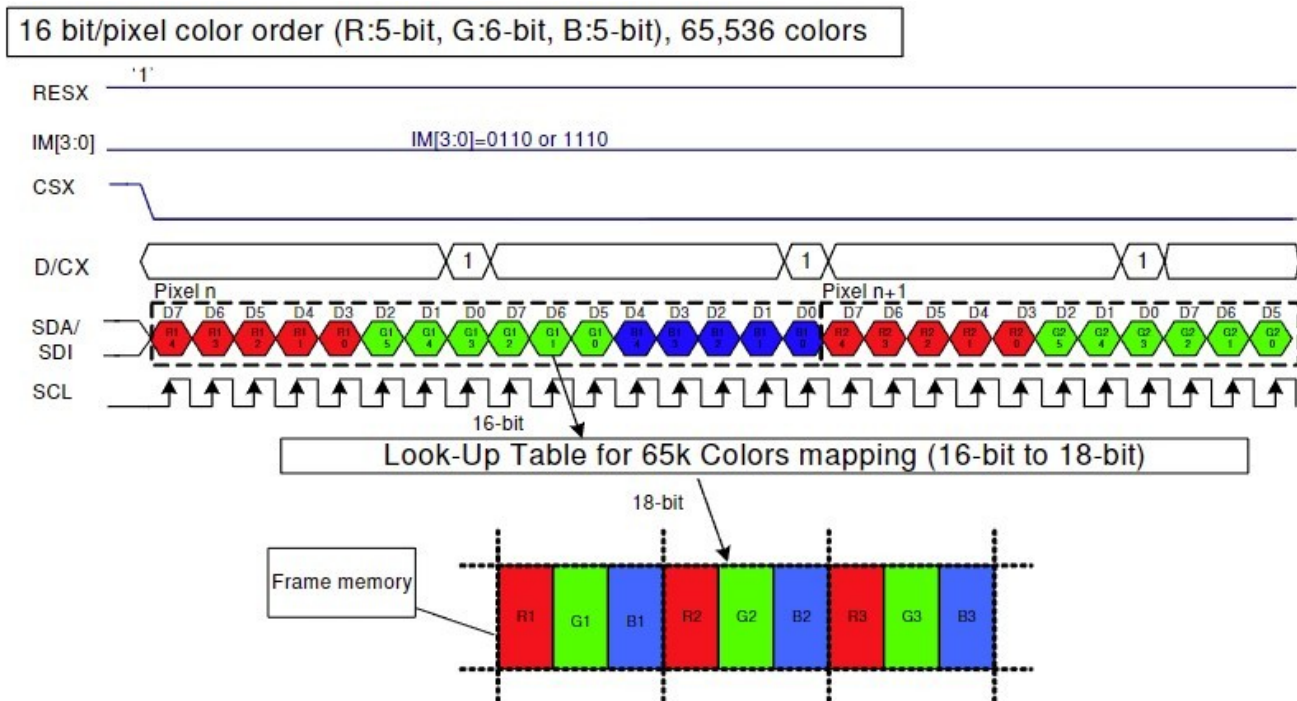
Протокол SPI

Передача команд и данных происходит по **SPI** в симплексном режиме. Каждый байт отправляется одной транзакцией. Сигнал **D/CX** обозначает является ли переданный байт командой (0) или данными (1)



Формат цвета

Формат RGB565: красный — 6 бит, зелёный — 6 бит, синий — 5 бит.



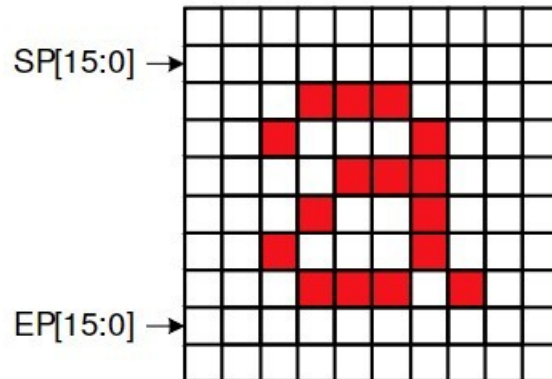
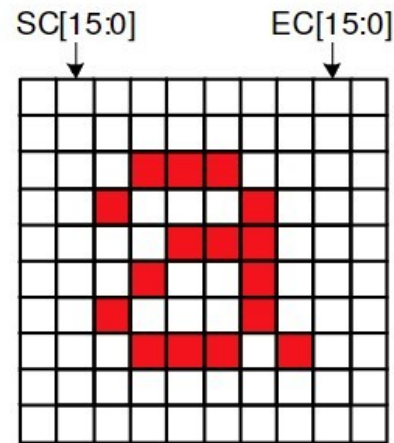
Команды

В даташите описано более 60 команд, мы используем только следующие:

Адрес	Название команды	Описание
0x01	SWRESET	Программный сброс
0x11	SLPOUT	Выход из режима сна
0x3A	COLMOD	Формат пикселя (RGB565)
0x36	MADCTL	Ориентация дисплея
0x29	DISPON	Включение дисплея
0x2A	CASET	Выбор столбца для заиси
0x2B	PASET	Выбор строки для записи
0x2C	RAMWR	Команда для начала записи пикселей

Обновление области дисплея

- **CASET (0x2A)** — задать диапазон столбцов
- **PASET (0x2B)** — задать диапазон строк
- **RAMWR (0x2C)** — начать запись пикселей. После команды — поток пиксельных данных



Дисплей и уровни

- **Прикладной:** рисование любых фигур: круга, квадрата, треугольника, линии, текста, отдельного пикселя.
- **Драйверный:** ответственность за корректную инициализацию и работу с параметрами драйвера дисплея через протокольный уровень
- **Протокольный:** определяется, как сигналы SPI и отдельных ножек задают смысл битов, проходящих по интерфейсу
- **Платформенный:** На этом уровне создаются платформенные колбэки для вызова функций чтения и записи в SPI, и управления GPIO-пинами RST, CS и D/CX.
- **Физический:** корректное управление физическими сигналами дисплея и подключение драйвера к микроконтроллеру

